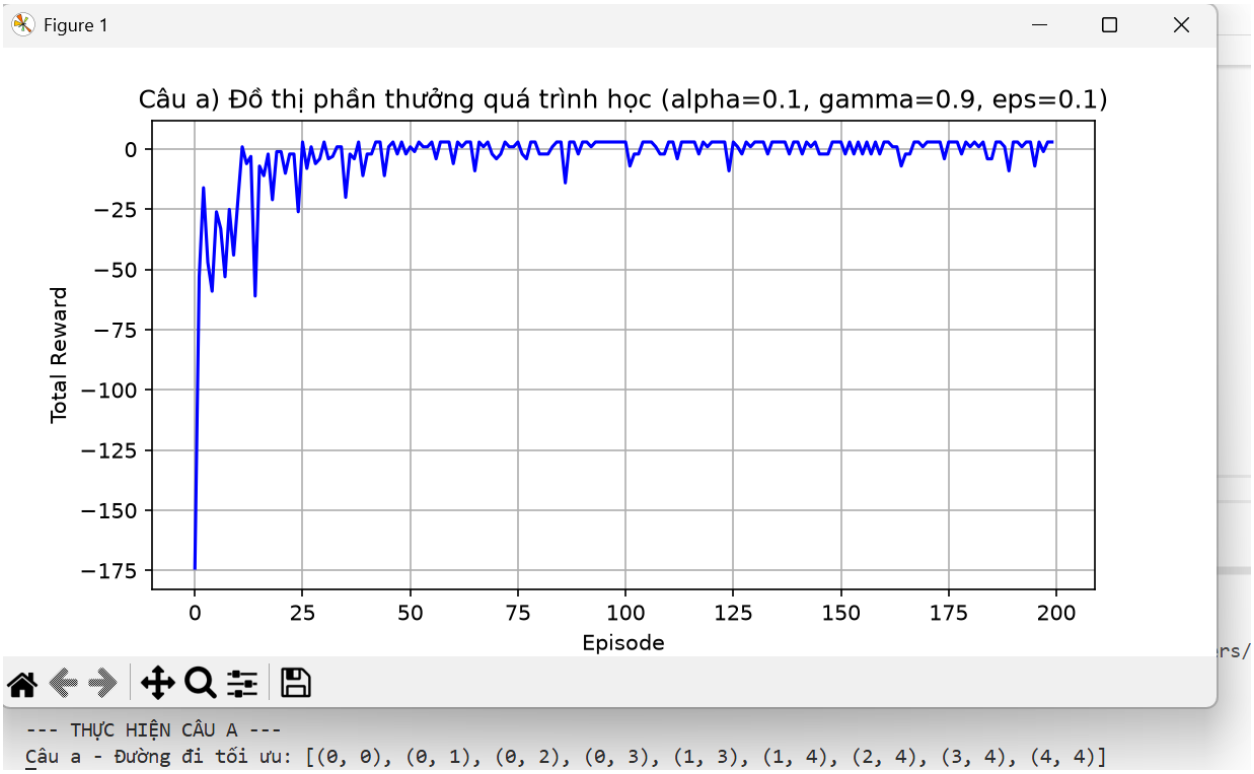
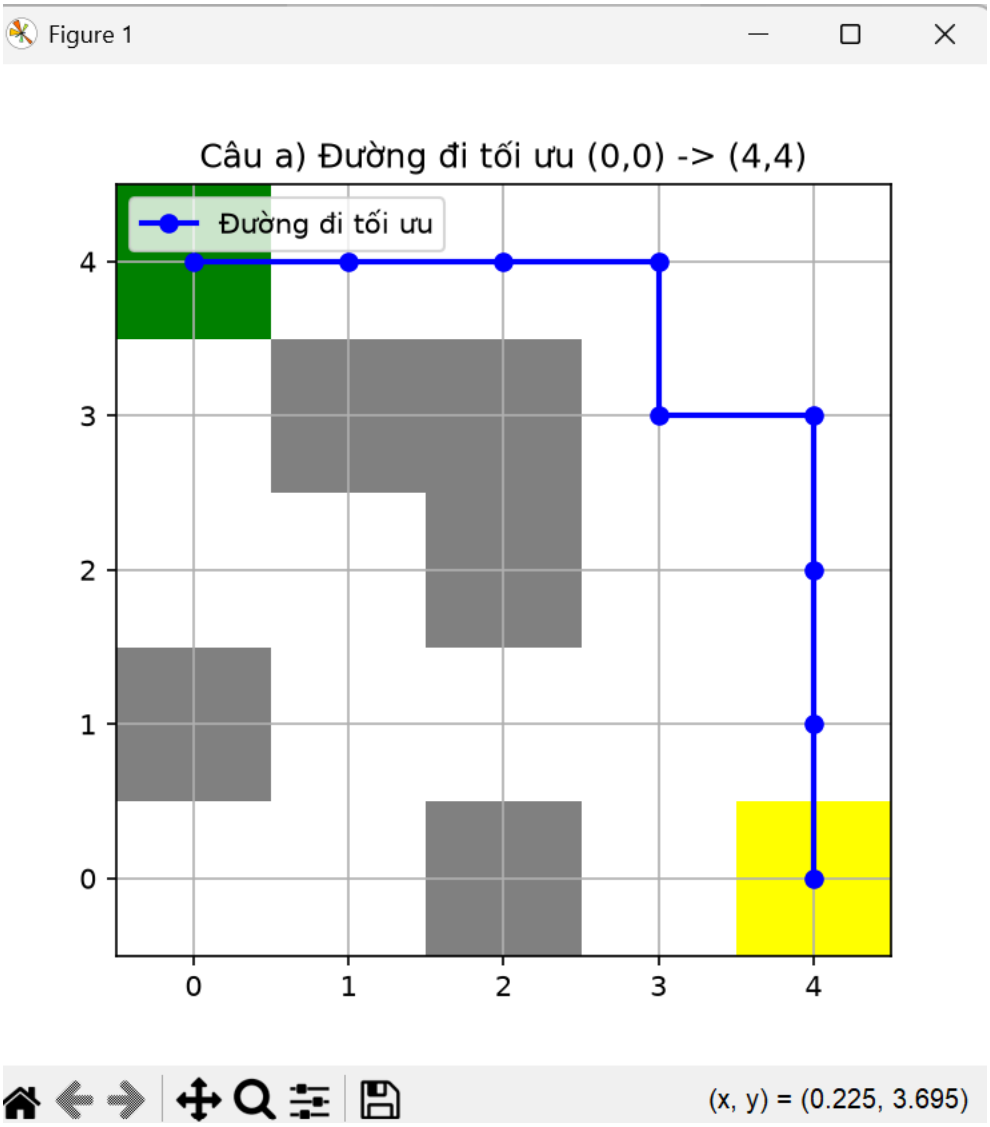
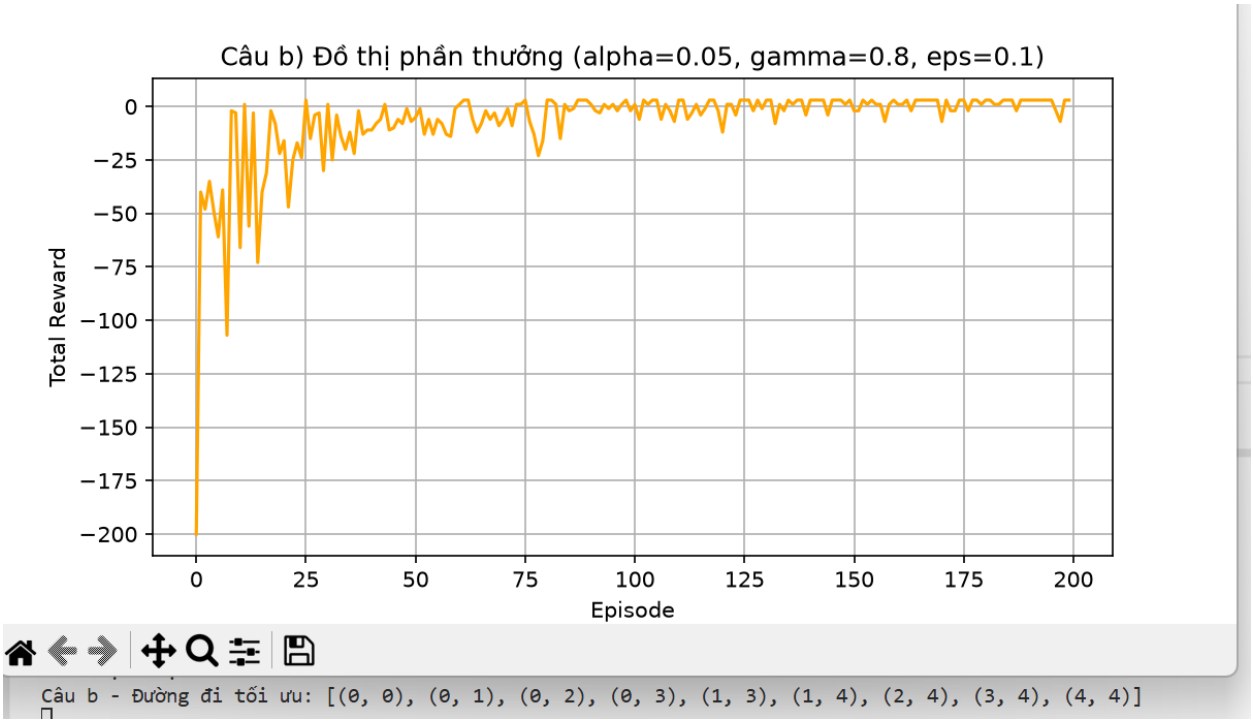
BÀI TẬP THỰC HÀNH BUỔI 2

a) đường đi tối ưu của bạn, đồ thị phần thưởng quá trình học.

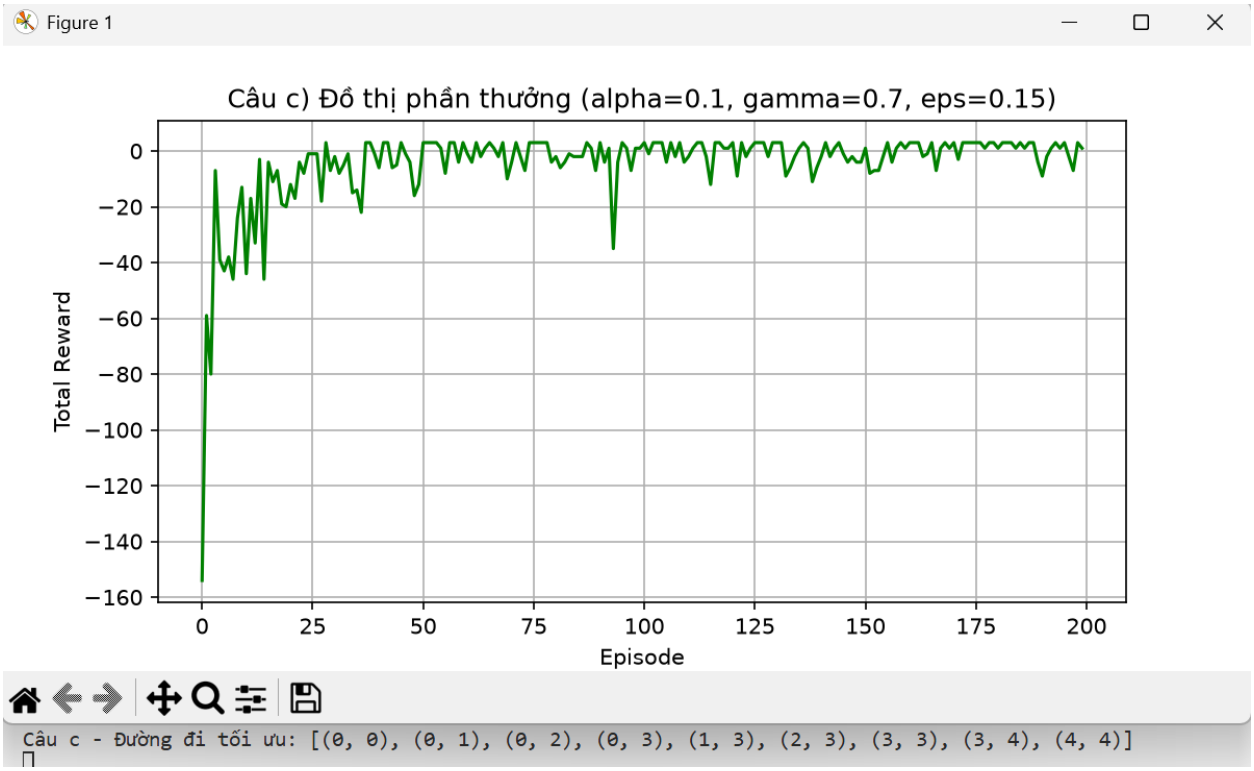




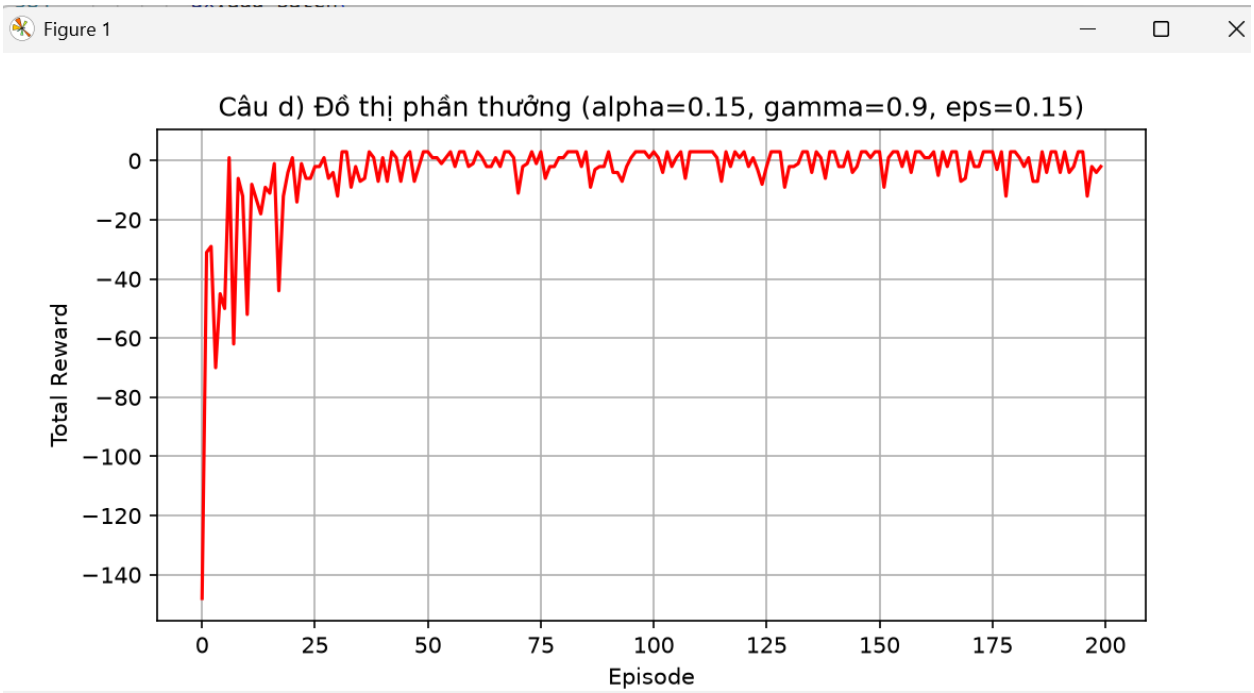
b) đường đi tối ưu, đồ thị phần thưởng với tốc độ học (alpha) 0.05, hệ số chiết khấu (gamma) 0.8, xác suất khám phá (epsilon) 0.1



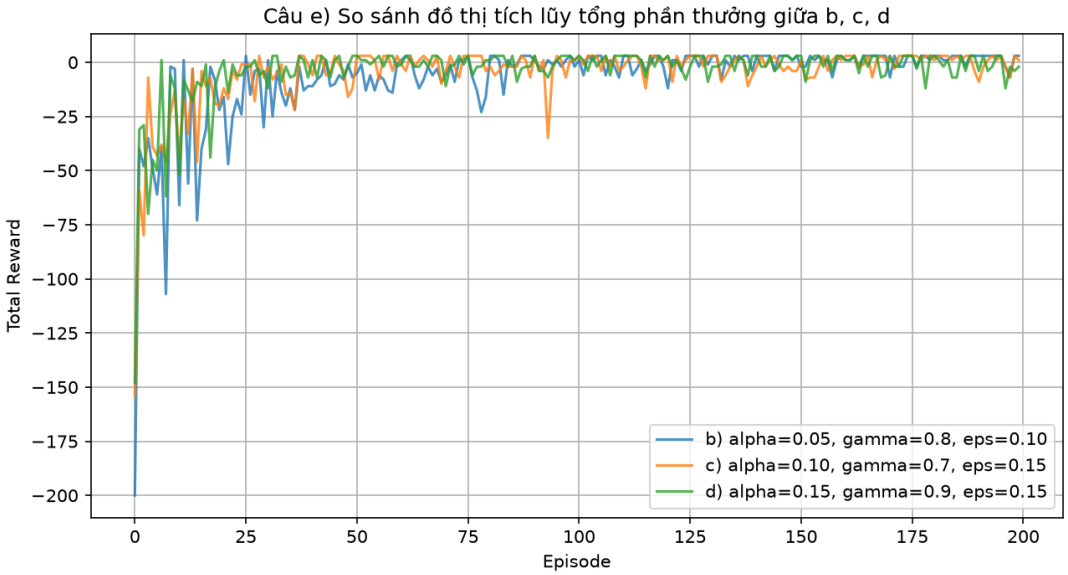
c) đường đi tối ưu, đồ thị phần thưởng với tốc độ học (alpha) 0.1, hệ số chiết khấu (gamma) 0.7, xác suất khám phá (epsilon) 0.15



d) đường đi tối ưu, đồ thị phần thưởng với tốc độ học (alpha) 0.15, hệ số chiết khấu (gamma) 0.9, xác suất khám phá (epsilon) 0.15



e) Giải thích sự khác nhau ở đồ thị tích luỹ tổng phần thưởng b,c,d



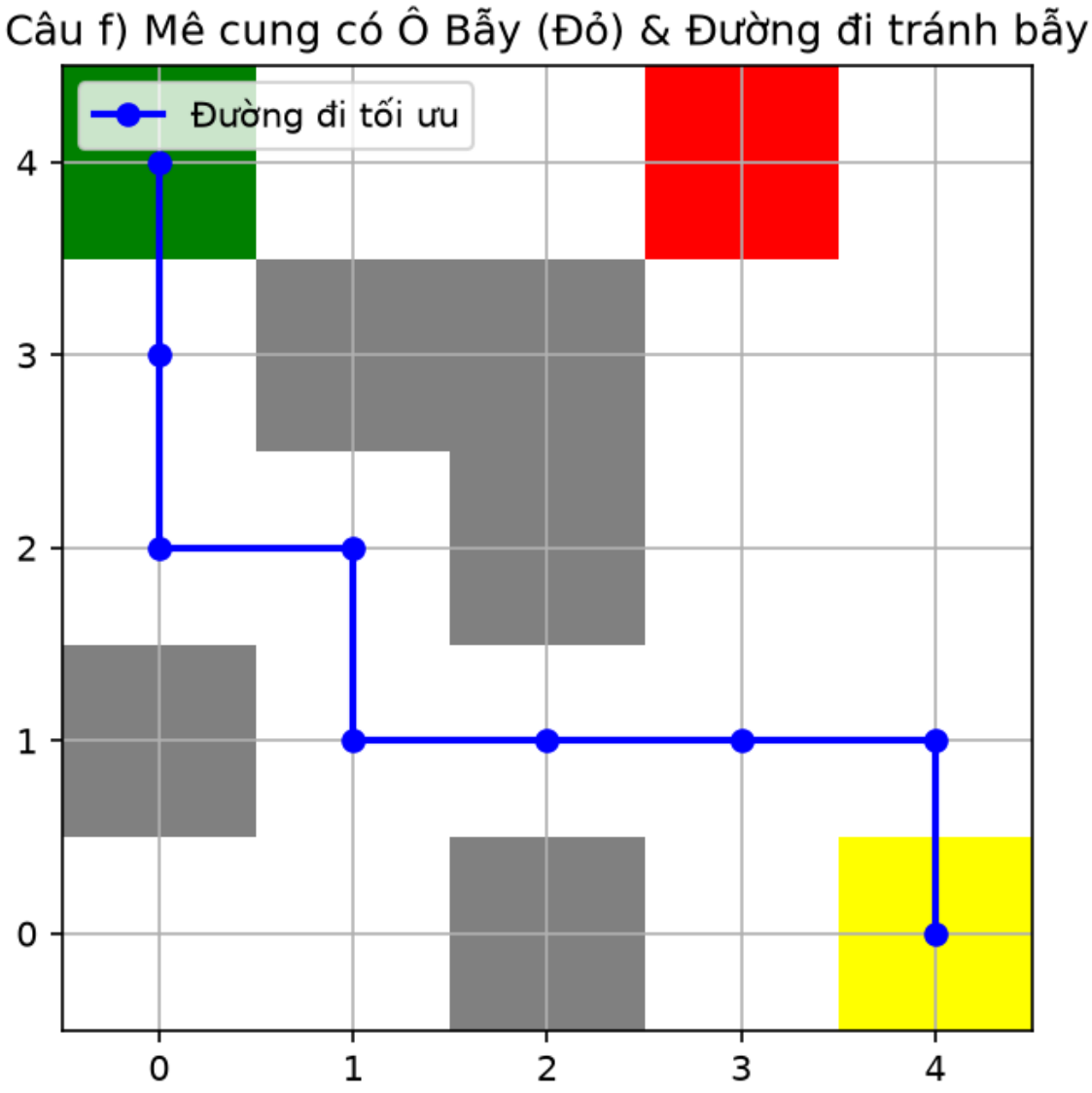
Câu b reward tăng lên khá chậm ở giai đoạn đầu nhưng lại càng trở nên ổn định hơn về sau. Vì learning rate = 0.05, tương đối thấp và hệ số gamma coi trọng các hành vi tương lai hơn

Câu c reward tăng nhanh hơn so với các chỉ số ở câu b nhưng lại dao động mạnh hơn rõ, và mức độ ổn định về sau cũng kém hơn so với b. Điều này có thể giải thích do ở c, ta đã tăng learning rate lên 0.1 và tăng hệ số exploration, agent sẽ khám phá nhiều hơn.

Câu d Cập nhật mạnh, coi trọng tương lai, reward có thể tăng khá nhanh nhưng đồng thời cũng dao đọng mạnh. Do cả lúc này cả hệ số learning rate và exploration đều cao.

f) hãy tạo 1 ô bẫy trên đường đi với phần thưởng là -30 và huấn luyện lại. In mê cung đã tạo bẫy, và in đường đi sau khi huấn luyện

```
Mê cung đã thêm bẫy (-30) tại ô (0, 3):
[[0 0 0 3 0]
 [0 1 1 0 0]
 [0 0 1 0 0]
 [1 0 0 0 0]
 [0 0 1 0 2]]
Câu f - Đường đi mới sau khi tránh bẫy: [(0, 0), (1, 0), (2, 0), (2, 1), (3, 1), (3, 2), (3, 3), (3, 4), (4, 4)]
```



g)

Thiết kế kết hợp đường xu hướng (Trend Line) với Học tăng cường (RL) của nhóm tác giả trên Tạp chí Khoa học Đại học Cần Thơ là RẤT HIỆU QUẢ VÀ CÓ TÍNH THỰC TIỄN CAO.

Thị trường tiền điện tử vốn mang tính biến động mạnh và chứa nhiều nhiễu. Nếu chỉ dùng RL thuần túy, agent dễ rơi vào tình trạng thử-sai ngẫu nhiên dẫn đến thua lỗ lớn.

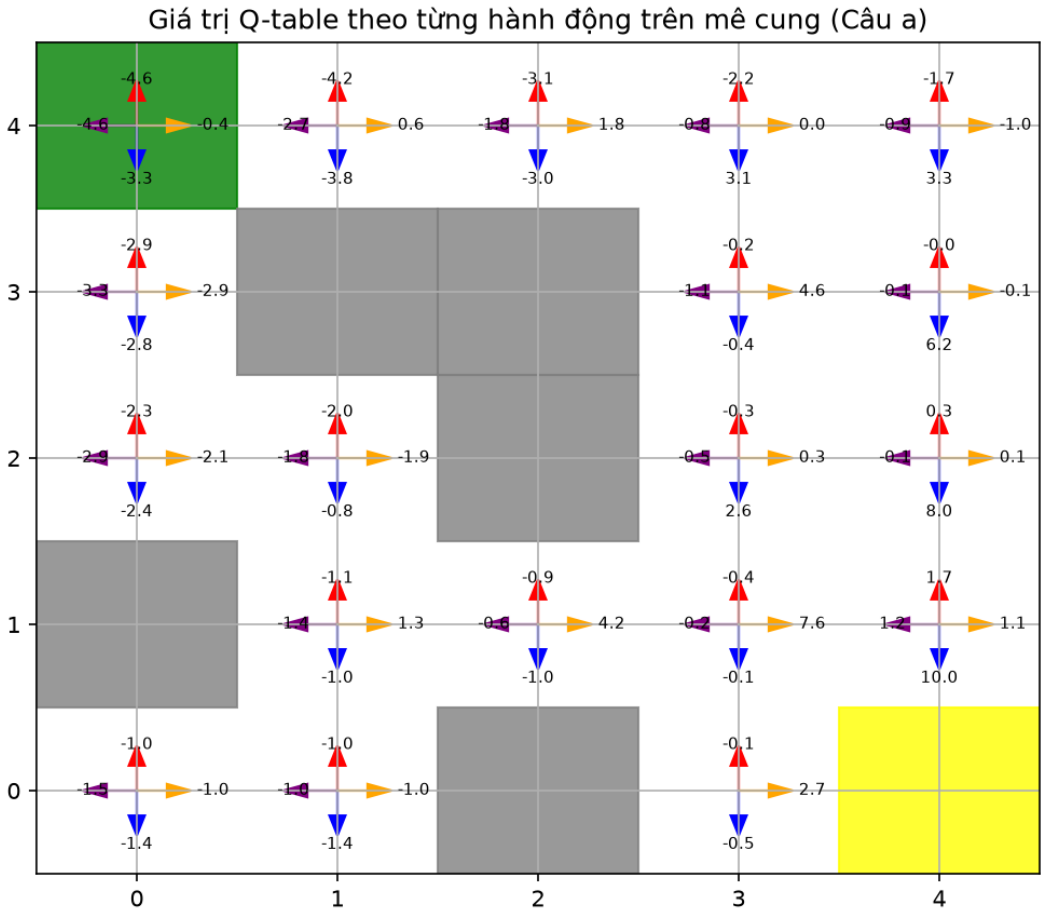
Việc tích hợp đường xu hướng đóng vai trò như một bộ lọc không gian hành động (Action Space Filtering), định hướng agent ưu tiên giao dịch theo xu hướng chủ đạo (Up-trend/Down-trend).

Kết quả thực nghiệm trên Bitcoin và Dogecoin chứng minh mô hình giúp cải thiện rõ rệt lợi nhuận hàng năm (Annual Profit) và giảm thiểu rủi ro sụt giảm tối đa (Maximum Drawdown).

i) Tính năng gợi ý phát triển thêm: mở rộng mê cung lớn hơn, thêm cổng dịch chuyển từ vị trí này qua vị trí khác, thêm điều kiện ràng buộc như phải kiếm được chìa khoá trước khi đến đích, tìm kiếm thêm đồ ăn/ vật phẩm/ kho báu trong mê cung hoặc có quái vật truy đuổi …

1. Mở rộng quy mô Mê cung & Deep Q-Network (DQN): Nâng kích thước từ 5x5 lên 20x20 hoặc 50x50,

kết hợp Mạng Nơ-ron sâu (DQN) để giải quyết vấn đề bùng nổ không gian trạng thái.



```python
import os
import matplotlib.patches as patches
import matplotlib.pyplot as plt
import numpy as np
from matplotlib.animation import FFMpegWriter, FuncAnimation

# Thiết lập đường dẫn FFMpeg nếu bạn có xuất video animation (không bắt buộc)
# plt.rcParams['animation.ffmpeg_path'] = r'C:\\Users\\Downloads\\ffmpeg-7.1.1-full_build\\bin\\ffmpeg.exe'

# Set seed để kết quả chạy ổn định và tái lập được
np.random.seed(42)

# ==============================================================================
# KHỞI TẠO MÔI TRƯỜNG MÊ CUNG & CÁC THAM SỐ CHUNG
# ==============================================================================
# Giá trị ô trong mê cung:
# 0: Ô trống (đường đi)
# 1: Tường
# 2: Mục tiêu (Goal)
# 3: Bẫy (Trap - dùng cho câu f)

maze_default = np.array(
    [
        [0, 0, 0, 0, 0],
        [0, 1, 1, 0, 0],
        [0, 0, 1, 0, 0],
        [1, 0, 0, 0, 0],
        [0, 0, 1, 0, 2],
    ]
)

start = (0, 0)
goal = (4, 4)
episodes = 200
max_steps = 100


# Chuyển tọa độ (x, y) thành chỉ số trạng thái 1D (0 đến 24)
def state_to_index(x, y):
  return x * 5 + y


# Hàm thực hiện hành động chuẩn khung sườn của thầy
def take_action(state, action, maze_env):
```

```
  x, y = state
  if action == 0:  # Lên
    next_x, next_y = x - 1, y
  elif action == 1:  # Xuống
    next_x, next_y = x + 1, y
  elif action == 2:  # Trái
    next_x, next_y = x, y - 1
  else:  # Phải
    next_x, next_y = x, y + 1

  # Kiểm tra ranh giới và tường
  if (
      next_x < 0
      or next_x >= 5
      or next_y < 0
      or next_y >= 5
      or maze_env[next_x, next_y] == 1
  ):
    return state, -5  # Va vào tường

  next_state = (next_x, next_y)

  # Kiểm tra các ô đặc biệt
  if maze_env[next_x, next_y] == 3:  # Ô Bẫy
    reward = -30
  elif next_state == goal:  # Ô Mục tiêu
    reward = 10
  else:  # Ô trống bình thường
    reward = -1

  return next_state, reward


# Hàm huấn luyện Q-Learning dùng chung cho tất cả các câu
def train_q_learning(
    alpha_val, gamma_val, epsilon_val, maze_env=maze_default, num_ep=episodes
):
  q_table = np.zeros((25, 4))
  total_rewards = []

  for episode in range(num_ep):
    state = start
    total_reward = 0
    for step in range(max_steps):
      # Epsilon-greedy
```

```
      if np.random.rand() < epsilon_val:
        action = np.random.randint(4)  # Khám phá
      else:
        s_idx = state_to_index(state[0], state[1])
        action = np.argmax(q_table[s_idx])  # Khai thác

      next_state, reward = take_action(state, action, maze_env)

      # Tích lũy phần thưởng
      total_reward += reward

      # Cập nhật Q-table theo công thức Q-Learning
      s_idx = state_to_index(state[0], state[1])
      ns_idx = state_to_index(next_state[0], next_state[1])

      q_table[s_idx, action] = q_table[s_idx, action] + alpha_val * (
          reward + gamma_val * np.max(q_table[ns_idx]) - q_table[s_idx, action]
      )

      state = next_state
      if state == goal:
        break
    total_rewards.append(total_reward)

  # Tìm đường đi tối ưu
  state = start
  path = [state]
  visited = set([state])
  for _ in range(max_steps):
    s_idx = state_to_index(state[0], state[1])
    action = np.argmax(q_table[s_idx])
    next_state, _ = take_action(state, action, maze_env)

    if next_state in visited and next_state != goal:
      break  # Tránh vòng lặp vô tận nếu bị kẹt
    path.append(next_state)
    visited.add(next_state)
    state = next_state
    if state == goal:
      break

  return q_table, total_rewards, path


# Hàm vẽ mê cung chuẩn khung của thầy
```

```
def plot_maze(ax, maze_env=maze_default, path=None, title="Mê cung"):
  ax.clear()
  ax.set_xlim(-0.5, 4.5)
  ax.set_ylim(-0.5, 4.5)
  ax.set_xticks(np.arange(5))
  ax.set_yticks(np.arange(5))
  ax.grid(True)

  # Vẽ các ô mê cung
  for i in range(5):
    for j in range(5):
      if maze_env[i, j] == 1:  # Tường
        ax.add_patch(
            patches.Rectangle((j - 0.5, 4 - i - 0.5), 1, 1, facecolor="gray")
        )
      elif maze_env[i, j] == 2:  # Mục tiêu
        ax.add_patch(
            patches.Rectangle((j - 0.5, 4 - i - 0.5), 1, 1, facecolor="yellow")
        )
      elif maze_env[i, j] == 3:  # Bẫy
        ax.add_patch(
            patches.Rectangle((j - 0.5, 4 - i - 0.5), 1, 1, facecolor="red")
        )
      elif (i, j) == start:  # Điểm bắt đầu
        ax.add_patch(
            patches.Rectangle((j - 0.5, 4 - i - 0.5), 1, 1, facecolor="green")
        )

  # Vẽ đường đi nếu có
  if path is not None:
    path_x = [p[1] for p in path]
    path_y = [4 - p[0] for p in path]
    ax.plot(
        path_x,
        path_y,
        color="blue",
        marker="o",
        linewidth=2,
        markersize=6,
        label="Đường đi tối ưu",
    )
    ax.legend(loc="upper left")

  ax.set_title(title)
```

```python
# CÂU A:
print("\n--- THỰC HIỆN CÂU A ---")
q_table_a, rewards_a, path_a = train_q_learning(
    alpha_val=0.1, gamma_val=0.9, epsilon_val=0.1
)
print("Câu a - Đường đi tối ưu:", path_a)

# Vẽ đồ thị phần thưởng câu a
plt.figure(figsize=(8, 4))
plt.plot(rewards_a, color="blue", linewidth=1.5)
plt.xlabel("Episode")
plt.ylabel("Total Reward")
plt.title("Câu a) Đồ thị phần thưởng quá trình học (alpha=0.1, gamma=0.9,
eps=0.1)")
plt.grid(True)
plt.show()

# Vẽ mê cung và đường đi câu a
fig, ax = plt.subplots(figsize=(5, 5))
plot_maze(
    ax,
    maze_default,
    path_a,
    title="Câu a) Đường đi tối ưu (0,0) -> (4,4)",
)
plt.show()


# CÂU B
# ============================================================================
print("\n--- THỰC HIỆN CÂU B ---")
q_table_b, rewards_b, path_b = train_q_learning(
    alpha_val=0.05, gamma_val=0.8, epsilon_val=0.1
)
print("Câu b - Đường đi tối ưu:", path_b)

plt.figure(figsize=(8, 4))
plt.plot(rewards_b, color="orange", linewidth=1.5)
plt.xlabel("Episode")
plt.ylabel("Total Reward")
plt.title("Câu b) Đồ thị phần thưởng (alpha=0.05, gamma=0.8, eps=0.1)")
plt.grid(True)
plt.show()
```

```python
# CÂU C

print("\n--- THỰC HIỆN CÂU C ---")
q_table_c, rewards_c, path_c = train_q_learning(
    alpha_val=0.1, gamma_val=0.7, epsilon_val=0.15
)
print("Câu c - Đường đi tối ưu:", path_c)

plt.figure(figsize=(8, 4))
plt.plot(rewards_c, color="green", linewidth=1.5)
plt.xlabel("Episode")
plt.ylabel("Total Reward")
plt.title("Câu c) Đồ thị phần thưởng (alpha=0.1, gamma=0.7, eps=0.15)")
plt.grid(True)
plt.show()



# CÂU D

print("\n--- THỰC HIỆN CÂU D ---")
q_table_d, rewards_d, path_d = train_q_learning(
    alpha_val=0.15, gamma_val=0.9, epsilon_val=0.15
)
print("Câu d - Đường đi tối ưu:", path_d)

plt.figure(figsize=(8, 4))
plt.plot(rewards_d, color="red", linewidth=1.5)
plt.xlabel("Episode")
plt.ylabel("Total Reward")
plt.title("Câu d) Đồ thị phần thưởng (alpha=0.15, gamma=0.9, eps=0.15)")
plt.grid(True)
plt.show()


# CÂU E:
print("\n--- THỰC HIỆN CÂU E ---")
plt.figure(figsize=(10, 5))
plt.plot(rewards_b, label="b) alpha=0.05, gamma=0.8, eps=0.10", alpha=0.8)
plt.plot(rewards_c, label="c) alpha=0.10, gamma=0.7, eps=0.15", alpha=0.8)
plt.plot(rewards_d, label="d) alpha=0.15, gamma=0.9, eps=0.15", alpha=0.8)
plt.xlabel("Episode")
plt.ylabel("Total Reward")
plt.title("Câu e) So sánh đồ thị tích lũy tổng phần thưởng giữa b, c, d")
plt.legend()
```

```
plt.grid(True)
plt.show()


# CÂU F
print("\n--- THỰC HIỆN CÂU F ---")
maze_trap = maze_default.copy()
maze_trap[0, 3] = 3

q_table_f, rewards_f, path_f = train_q_learning(
    alpha_val=0.15, gamma_val=0.9, epsilon_val=0.15, maze_env=maze_trap
)

print("Mê cung đã thêm bẫy (-30) tại ô (0, 3):")
print(maze_trap)
print("Câu f - Đường đi mới sau khi tránh bẫy:", path_f)

fig, ax = plt.subplots(figsize=(5, 5))
plot_maze(
    ax,
    maze_trap,
    path_f,
    title="Câu f) Mê cung có Ô Bẫy (Đỏ) & Đường đi tránh bẫy",
)
plt.show()


# VẼ BẢNG SUMMARY TỔNG HỢP VÀ BẢNG Q-TABLE CHUẨN FORM CỦA THẦY

fig, ax = plt.subplots(figsize=(8, 8))


def draw_q_values(ax, q_table_env, maze_env):
  ax.set_xlim(-0.5, 4.5)
  ax.set_ylim(-0.5, 4.5)
  ax.set_xticks(np.arange(5))
  ax.set_yticks(np.arange(5))
  ax.grid(True)

  for i in range(5):
    for j in range(5):
      if maze_env[i, j] == 1:  # Tường
        ax.add_patch(
            patches.Rectangle(
                (j - 0.5, 4 - i - 0.5), 1, 1, color="gray", alpha=0.8
```

```
            )
        )
    elif maze_env[i, j] == 2:  # Mục tiêu
        ax.add_patch(
            patches.Rectangle(
                (j - 0.5, 4 - i - 0.5), 1, 1, color="yellow", alpha=0.8
            )
        )
    elif (i, j) == start:
        ax.add_patch(
            patches.Rectangle(
                (j - 0.5, 4 - i - 0.5), 1, 1, color="green", alpha=0.8
            )
        )

    if maze_env[i, j] != 1:
        s = state_to_index(i, j)
        q = q_table_env[s]
        x, y = j, 4 - i

        if q[0] != 0:  # Lên
            ax.arrow(x, y, 0, 0.15, head_width=0.08, color="red")
            ax.text(x, y + 0.25, f"{q[0]:.1f}", ha="center", fontsize=7)
        if q[1] != 0:  # Xuống
            ax.arrow(x, y, 0, -0.15, head_width=0.08, color="blue")
            ax.text(x, y - 0.35, f"{q[1]:.1f}", ha="center", fontsize=7)
        if q[2] != 0:  # Trái
            ax.arrow(x, y, -0.15, 0, head_width=0.08, color="purple")
            ax.text(x - 0.3, y, f"{q[2]:.1f}", va="center", fontsize=7)
        if q[3] != 0:  # Phải
            ax.arrow(x, y, 0.15, 0, head_width=0.08, color="orange")
            ax.text(x + 0.3, y, f"{q[3]:.1f}", va="center", fontsize=7)


draw_q_values(ax, q_table_a, maze_default)
ax.set_title("Giá trị Q-table theo từng hành động trên mê cung (Câu a)")
plt.show()

print("\n=== HOÀN THÀNH TOÀN BỘ CHƯƠNG TRÌNH TỪ CÂU A ĐẾN CÂU I ===")
```